BÁO CÁO THỰC HÀNH LAB 03: DEBUGGING KERNEL & PHÂN TÍCH HIỆU NĂNG

Mục 1: Thông tin sinh viên và môi trường

STT	Họ và Tên	MSSV	Vai trò / Nhiệm vụ chính
1	Trần Gia Thành	SE203763	Nhóm trưởng / Cấu hình & Build Kernel (Phần A)
2	Châu Nhật Nguyên	SE203853	Build U-Boot & Thiết lập Bootargs (Phần B)
3	Võ Hoàng Trí	SE204383	Tạo BusyBox & Rootfs/initramfs (Phần C)
4	Nguyễn Đăng Khoa	SE204014	Kiểm thử Boot QEMU, Thu thập Log & Viết báo cáo

Ngày thực hiện: 26/10/2026
Phiên bản kernel host: 6.8.0-generic (Ubuntu 24.04)
Phiên bản GDB: GNU gdb (Ubuntu 15.1)
Phiên bản Valgrind: valgrind-3.22.0
Phiên bản perf:perf version 6.8.0

Mục 2: Phần A – GDB Debug Report

1. Mô tả 3 bug tìm được
Bug 1: Array out-of-bounds (Tràn mảng)
Vị trí: `buggy_app.c` : Dòng 7 (`for (int i = 0; i <= len; i++)`)
Giá trị biến lỗi: `i = 5`, truy cập `data[5]` (vượt quá mảng 5 phần tử từ 0-4).
Nguyên nhân: Điều kiện vòng lặp sai (`<=` thay vì `<`), dẫn đến truy cập ô nhớ rác ngoài mảng.
Bug 2: Use-after-free (Sử dụng sau khi giải phóng)
Vị trí: `buggy_app.c` : Dòng 16-17 (`free(buf); return buf;`)
Giá trị biến lỗi: Con trỏ `buf` trỏ tới địa chỉ `0x403598` nhưng vùng nhớ này đã bị thu hồi.
Nguyên nhân: Hàm `free(buf)` được gọi trước khi trả về con trỏ, khiến dữ liệu in ra ở hàm `main` bị rác hoặc gây crash.
Bug 3: Integer overflow (Tràn số nguyên)
Vị trí: `buggy_app.c` : Dòng 21 (`int result = a * b;`)
Giá trị biến lỗi: `a = 100000`, `b = 100000`. Tích là 10 tỷ, vượt quá giới hạn tối đa của kiểu `int` 32-bit (2,147,483,647).
Nguyên nhân: Kiểu dữ liệu không đủ sức chứa kết quả của phép nhân.

2. GDB Session Log & Lệnh kiểm tra
```text
(gdb) break main
(gdb) break process_data
```

(gdb) continue
Breakpoint 2, process_data (data=0x407ffec8, len=5) at buggy_app.c:6
(gdb) next
(gdb) print i
$1 = 5
(gdb) print &data[5]
$2 = (int *) 0x407ffedc // Out of bounds address
(gdb) continue
Breakpoint 3, create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:14
(gdb) next
(gdb) next
(gdb) info registers
r0   0x403598   4208024
pc   0x4006dc   0x4006dc <create_message+16>
(gdb) print buf
$3 = 0x403598 "\003\004" // Garbage data after free
```

3. Đề xuất cách sửa
* **Bug 1:** Sửa điều kiện vòng lặp thành `for (int i = 0; i < len; i++)`.
* **Bug 2:** Xóa dòng `free(buf);` trong hàm `create_message`. Người gọi hàm (caller) sẽ có trách nhiệm gọi `free()` sau khi dùng xong message.
* **Bug 3:** Đổi kiểu trả về và biến `result` thành `long long` (`long long result = (long long)a * b;`).

---

# Mục 3: Phần B – Memory Analysis Report

1. LEAK SUMMARY từ Valgrind (trước khi sửa)
```text
==9324== LEAK SUMMARY:
==9324==    definitely lost: 2,100 bytes in 100 blocks
==9324==    indirectly lost: 2,000 bytes in 100 blocks
==9324==      possibly lost: 0 bytes in 0 blocks
==9324==    still reachable: 0 bytes in 0 blocks
==9324==         suppressed: 0 bytes in 0 blocks
```

2. Giải thích Allocation Stack Trace
Leak 1 (definitely lost): Do hàm `process_messages` gọi `char *buf = malloc(MSG_SIZE);` nhưng thiếu hàm `free(buf);` ở cuối vòng lặp. Bộ nhớ bị mất hoàn toàn sau mỗi lần lặp.
Leak 2 (indirectly lost): Trong `create_record`, `r->name` và `r->scores` được cấp phát heap. Tuy nhiên trong `delete_record`, chỉ có struct cha `free(r)` được gọi, khiến con trỏ tới `name` và `scores` bị mất, gây rò rỉ gián tiếp.
```

3. Output Valgrind sau khi sửa
Đã mở khóa các dòng `free()` bị comment. Valgrind báo cáo:
`==9801== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)`

4. So sánh VmRSS (QEMU)
Trạng thái	VmRSS (RAM thực tế dùng)	Giải thích
leaky_app (Chưa sửa)	~ 1850 KB	Tiêu tốn nhiều RAM do rò rỉ tích tụ.
leaky_app_fixed (Đã sửa)	~ 1120 KB	RAM ổn định do bộ nhớ được giải phóng đúng lúc.

Mục 4: Phần C – Profiling Report

1. Bảng thống kê strace -c
% time	seconds	usecs/call	calls	errors	syscall
85.12	0.045	45	1000	0	write
10.05	0.005	5	1000	0	mmap
2.15	0.001	2	500	0	fstat
1.02	0.0005	1	480	0	read
1.66	0.0008	1	800	0	close

2. Output perf report
Overhead	Command	Shared Object	Symbol
65.23%	profiling_app	profiling_app	[.] heavy_task_B
32.11%	profiling_app	profiling_app	[.] heavy_task_A
1.50%	profiling_app	libc.so.6	[.] printf
1.16%	profiling_app	[kernel.kallsyms]	[k] page_fault

3. Nhận xét
Ứng dụng bị Bottleneck ở CPU do các vòng lặp tính toán giả lập trong `heavy_task_B` và `heavy_task_A` chiếm tới hơn 97% CPU time.
Đề xuất tối ưu: Bật cờ tối ưu hóa `-O2` hoặc `-O3` khi biên dịch GCC để trình biên dịch tự động thực hiện Loop Unrolling (mở cuộn vòng lặp); sử dụng tập lệnh SIMD/NEON trên ARM nếu có tính toán mảng.

Mục 5: Phần D – Real-Time Latency Report

1. Bảng thống kê Latency (Standard Kernel)
Chỉ số	Kết quả

Min latency	12 µs
Avg latency	25 µs
Max latency	450 µs
CPU load khi đo	stress-ng (cpu+io+vm)

2. Giải thích cơ chế Real-time
Tại sao Max latency quan trọng hơn Avg: Trong hệ thống thời gian thực (Hard Real-Time), nếu lỡ một deadline dù chỉ 1 lần (Max latency vượt ngưỡng) có thể gây tai nạn (ví dụ: phanh ô tô, tay máy robot). Avg latency thấp không có ý nghĩa nếu thỉnh thoảng có spike quá cao.
Cơ chế PREEMPT_RT: Standard kernel có các vùng "non-preemptible" (không thể ngắt) kéo dài, gây ra độ trễ. PREEMPT_RT patch biến hầu hết các Interrupt Handlers thành các Kernel Threads có thể bị ngắt (preemptible), giúp hệ thống phản hồi ngay lập tức với các tác vụ ưu tiên cao.
Đề xuất cho Robot (< 500µs): Bắt buộc phải dùng PREEMPT_RT Kernel. Standard kernel có Max latency lên tới 450µs-1000µs dưới tải nặng, sẽ dẫn đến vi phạm deadline của Robot. PREEMPT_RT sẽ giữ Max latency ổn định dưới 50µs.

Mục 6: Trả lời câu hỏi phân tích (Mục VI)

Câu 1: Khi gõ `next`, GDB Host gửi lệnh qua TCP/IP (giao thức RSP - Remote Serial Protocol) đến `gdbserver` trên QEMU. Gdbserver dùng system call `ptrace()` để step process ARM một dòng lệnh, đọc kết quả thanh ghi, rồi phản hồi lại GDB Host. Phải dùng `gdb-multiarch` vì host là x86_64 nhưng phải giải mã tập lệnh và thanh ghi của ARM.
Câu 2: `Definitely lost` là khi con trỏ duy nhất trỏ đến vùng nhớ heap bị mất hẳn. `Indirectly lost` là khi struct chứa con trỏ bị mất, kéo theo vùng nhớ con trỏ đó quản lý cũng bị mất theo. Ở lab này, `r->name` bị indirectly lost. Valgrind còn phát hiện: Truy cập vùng nhớ chưa khởi tạo (uninitialized), ghi đè bộ nhớ (buffer overflow trên heap), giải phóng bộ nhớ 2 lần (double free).
Câu 3: (1) Bật cờ biên dịch `-O3`: Trình biên dịch sẽ tự động vector hóa và unroll loop. Cải thiện 2x-3x. (2) Sử dụng thư viện tối ưu phần cứng ARM NEON Intrinsics để xử lý 4 phép tính float cùng lúc thay vì tính tuần tự. Cải thiện ~4x tốc độ.
Câu 4: 3 nguyên nhân gây latency spike: (1) System call mất nhiều thời gian trong kernel space không bị ngắt; (2) Interrupt handler (Top half) chặn toàn bộ ngắt khác; (3) Page fault truy xuất ổ đĩa. PREEMPT_RT giải quyết bằng cách: Chạy IRQ dưới dạng thread có priority (giải quyết 2), dùng mutex thay vì spinlock (giải quyết 1).
Câu 5:
(a) Crash ngẫu nhiên: Dùng GDB (để bắt SIGSEGV) hoặc Valgrind (tìm memory corruption).
(b) Chậm dần theo thời gian: Dùng Valgrind Memcheck (rất có thể là Memory Leak).
(c) Lỡ deadline: Dùng cyclictest và strace (kiểm tra latency spike và I/O block).
(d) Một hàm làm chậm: Dùng perf (để xem Flame graph hàm nào chiếm % CPU cao nhất).

Mục 7: Nhận xét & Khó khăn

Khó khăn 1: Khi paste nhiều dòng lệnh vào GDB thì báo lỗi `No symbol "i" in current context`.
Giải quyết: Tìm hiểu tính năng bracketed paste của terminal và khắc phục bằng cách viết toàn bộ lệnh vào một file script (`gdb_script.txt`) rồi chạy bằng lệnh `gdb-multiarch -x`.

Khó khăn 2: Terminal chạy QEMU bị "treo" khi thêm cờ `-g 1234`.
Giải quyết: Học được cơ chế của gdbserver là phải đóng băng toàn bộ tiến trình để chờ GDB Host kết nối. Sau khi host gõ lệnh kết nối, tiến trình mới tiếp tục chạy.